

26.08.24

From the OOD eval of afucm we can see it adapts to size

Additional setups

- Adjust size randomization range to [0.8, 1.5], test range [0.6, 0.8) + (1.5, 1.8]
- Use aux supervision on grasp width, weight tuned later
- Use mesh generated from the image-to-3d model from images for training. 10 instances per object category

Also, it is time to test on different objects. Can you use the 3mm cube as the nominal tofu block and launch a 650 demo collection run, then train with afucm and eqrth setup? No real data presents so pure sim is ok. Smoke test data collection with 40 episodes and generate all videos to let me verify things, once I give an OK, then you proceed with rest of the data collection and training.

26.08.23

/nobackup/proj/disk/softenable-codesign26/personal/ikemura/gentle_manip/logs/dppo/dppo-pret
rain/single_lift_mushroom_real_abs_cmd/qjzsf
and dp3 result works pretty nice in sim

Real demo trained policy motion look much cleaner, maybe we should aim that for sim demo

Top right corner is real policy weakness

qjzsf real only run has ok success in sim, but as we expected the stress is very high,

Todo summary:

1. Collect a set of real world data with a 3cm cube placed right below the arm, and analyze data difference in sim and real. - local agent
2. Analyze how real-world demo differs from in-sim demo, like trajectory characteristics, speed, grasp speed, grasp width etc. Try to match in-sim scripted demo to real demo's properties. This may make cotraining better - local agent
3. Ablation of afucm with different real data amount, I want to test them in real world. Test 1demo, 5demos, 10demos, 20demos, 30 demos. Currently running by cluster agent.
4. Sync with colleague the fixed setup
 - a. Native 7d action space (euler for rotation)
 - b. Arm focus pnt cld
 - c. Quaternion for rot proprioception
 - d. Real-world workspace DR
 - e. Use DPPO codebase

- f. x3.5 big net: visual_feature_dim 512 + mlp_dims [1024, 1024, 1024] (2.89M EMA params)
- 5. Reduce occlusion in the generated demo, either use occlusion penalty in grasp synthesis or hard constrain the angles - local agent
- 6. Add more mushroom variants that are closer to real-world mushroom shapes - myself
- 7. Have an OOD size+shape test scenario - cluster agent
- 8. Try to improve robustness to missed grasps, if possible - local agent
- 9. Promote everything to generalist - cluster agent for end-to-end experiments
- 10. Test with more gentle grasp (small or no over squeeze, see if policy still works, with real-world cotrain added) - cluster agent
- 11. Since success is starting to be stable, we can start Gentleness-aware model selection.
- 12. Memory in the policy - cluster agent
- 13. Aux objectives on the real data cotrain - cluster agent
- 14. Camera-pose DR, very slightly, maybe 0.5 cm in each axis, and 1-5 degrees. - cluster agent

Plan, take care 1 and 2 first, 3 is ongoing, proper ablation that goes into the paper will be done on the finalized method. 4 should be done immediately on the next working day. 5 can be added to the new version of synthesis after changes with 2 confirmed working. 6 can be done in parallel (the mesh preparation part, in the end we just run the good pipeline with more meshes). 7 can be developed and tested in parallel, using previously trained policies as test cases. 8 deferred until everything else checks out. 9 is the last, after 8 is decided (do or not to do). 10 should be done before 8 but after changes from 2 and 5 are confirmed to be working. Tho 10 can also be done with afucm on cluster. 11 can be done from now on. 12 can be tested on the current afucm setup on the cluster. 13 too, can be tested on afucm. 14 can also be tested on afucm.

26.08.22

- Any training speedup methods?
- hwo can be reproduced, armfocus if fine
- Cotraining is on its way of training, early evals seems fine (in sim)
 - Tried oversampling and no oversampling
- Check if in nmbtz (single_lift_mushroom_soft_armfocus_realws_cmd) the failed ones are all the smaller mushrooms
- Seems hwo pipeline is the best we get so far, and we can fix the following cfigs
 - Abs 7d command
 - Realworkspace DR
 - Quat obs
 - Arm focus pnt cld

! Analyze difference in demo between real and sim

! Analyze pure arm pnt cld and proprio difference in sim and real

! Do a demo number experiment? Not sure meaningful on a single specialist

! Run all the good policies in sim to real (including the cotrained one)

/nobackup/proj/disk/softenable-codesign26/personal/ikemura/gentle_manip/third_party/DP3/3D-Diffusion-Policy/data/outputs/single_lift_mushroom_real_delta-dp3-realabl_seed42/sim_eval_harness/latest/render/batch10_env4.mp4 seems cool, real world data dp3 performing somewhat non degenerative in sim.

Tomorrow, plan carefully what to do next

26.08.21

- /nobackup/proj/disk/softenable-codesign26/personal/ikemura/gentle_manip/logs/dppo/dppo-pretrain/single_lift_mushroom_soft_hwo_7d_cmd/eibno/eval/state_100_eval has 0.86
- Found that the randomization range was $x \in [0.425, 0.515]$, $y \in [-0.045, +0.045]$, not good for real deployment. I'd change to $[0.3, 0.48]$ for x and $[-0.11, 0.11]$ for y

Need to run the real-world trained policy to see the performance

Also need to adjust the randomization range

For abs cmd add delta limits

26.08.20

Decided to record a demo in real-world (55), serving some purposes:

1. Check if absolute action or relative is better
 2. A baseline
 3. Check action smoothness and evolution (can also do sim2real gap analysis)
 4. Can be used for co-training later
- At least in the current setup, non of the auxiliary objectives (rot not tested) help.
 - Now I see training seems to have high variance (with data collection too), analyze
 - The armfocus policy with sdf (vqgsn) performed ok on real printed mushroom, less good on real mushrooms
 - Found out in real the z touching the table is -3.5mm, might be why policy struggled to pick up because the mushroom is at least 3.5 mm lower than in sim.

hwo with quaternion performed quite nice, but real world deployment is suboptimal, could be the absolute action space problem. Not sure if hwo can be converted to the 7d absolute action and cotrain with real world data.

Another question is how hwo is different from the real points? That might hint us towards sim2real gap.

Also how hwo is different from other dataset? Why did it have a very high success but not others?

Well, at least we have a policy that performs ok in sim (hwo-quat one), we can try to build based on that to aim for a good policy that also works ok in real.

I should still be careful of the potential sim2real failure of arm focus.

The immediate next step is to recollect hwo with 7d

One concern is that we might later need to recalibrate the camera, which means we need to recollect the data in real (or not? is there a easy conversion?)

26.08.19 - new

- Testing with point focus weight. Running
 - Cho with point focus, then train the same ahaxs setup
 - FEM with point focus, 2mm extra grasp, then train the same ahaxs setup
 - Hypothesis: may help sim policy, but may create even larger sim2real gap

Ruiyu mentioned some tricks to make the policy smooth, not solving for sudden jerks maybe, that may need us to look into more on how to generate demo that mimics human motion (smooth)

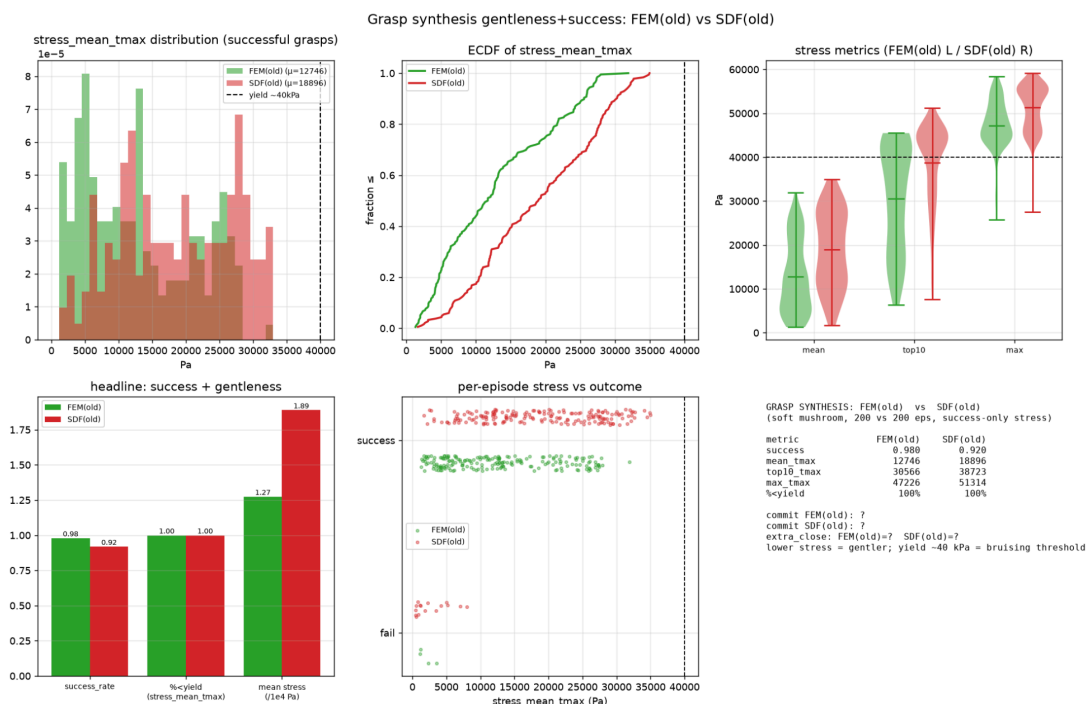
Testing four things:

1. Focus weight
 - a. With SDF synth, (vqgsn) has 0.6 success rate at 400, higher than any 400 before, maybe a good sign? Or just saturate there. Anyways, convergence faster is sure.
2. Aux objective 1: contact pred
3. Aux objective 2: object position regression
4. Aux objective 3: object pose regression

Since bwvei performed nice in sim, we can go with its setup and try to ablate. Tho that was using rot6d and I think that harmed sim2real so I would this time use quat. A todo is to train a bwvei with quat and try directly applying it in real world.

Summary of current status as of 26.08.19

- Grasp synthesis using FEM is complete, we can achieve okish policy performance with 650 demos collected from it if we include 5mm extra grasp, otherwise success is low (0.2). The main problem is that the policy can't learn precise and gentle grasp width and may miss the lift timing. Also robustness is not there as we don't do any retries in demo.
- Comparing FEM based and SDF based grasp synthesis, the former is for sure more gentle, see the figure below



- Sim2real is ok but not perfect, there are still gaps, the gripper could miss the grasp both due to moving to a slightly wrong position and wrong grasp width (latter existing in sim too)
- Tried rot6d representation but seems it gives poor sim2real result, we may just use quaternion as it is the setup of the best sim2real policy now (ahaxs run)
- My colleague developed cross-category grasping by merging demos collected on multiple types of objects and learning a single policy. The problem of not learning the precise grasp is still somewhat an issue, though the success rate is okish.

So all-in-all, the two main issues we face now are:

- Despite having good demonstrations, the policy can't learn to that level, it could be observation problems, training parameters setting or something else.
- Sim2real still has room for improvement, but I currently have no ideas other than trying to collect some real-world samples and co-train with simulation demos (this may not help, just some ideas to try out)

3. Absolute actions make policy jerky, currently addressed by EMA on action chunks and also with delta caps. The policy works ok in real, but I feel like this is a hack and maybe challenged by reviewers? (or not)
4. We've been testing in real with 3D printed objects that come from the simulation, this was to mitigate the shape sim2real gap. But eventually we need to test on real food, which may not have the exact shape covered by simulation randomized objects, we might need to edit the models manually to match shapes as close to real-world foods as possible (or 3D scan).

Some miscellaneous todos include:

1. Our in-sim evaluations are in-domain, not out-of-domain evaluation pipeline exists, and I don't have a very good idea on how, because we have many randomization parameters like object location, orientation, size, shape, stiffness.
2. Test with real-world demo with absolute actions
3. More initial pose randomization to promote retry (num steps proportional to the distance to the object)

Moving forward:

I don't have a very good idea on what level should we pursue now, the project deadline comes in 4 weeks, and we have to be careful on not to dig on problems that are too time-consuming to solve. If 80% success rate of a specialist is something we should accept, we could just solve for sim2real part. But it is also true that the policy did not fully learn gentleness, which might be detrimental later even if we solve sim2real.

2026.08.19

- Rot6d seems not good? Could be also a source of sim2real gap. Let's not use it for now.
- Recollected cho (jll) is worse than cho on the same training setup
- Seems jll has more gentle closing than cho, cho also has many episodes where the gripper is closed hard.

metric	cho	jll	Δ (jll - cho)
Pre-close (open) width	79.18 ± 0.02	79.18 ± 0.01	~ 0
Minimum width (tightest point)	35.95 ± 5.81 [10.29–50.69]	37.09 ± 5.52 [24.49–52.01]	+1.13mm looser
Hold-phase width (final grip)	35.96 ± 5.80	37.09 ± 5.52	+1.13mm
Closing amount (open - min)	43.23 ± 5.80	42.10 ± 5.51	-1.13mm less closed

Two things stand out:

1. **The mean shift is small but real and consistent** — every width metric (min/final/hold) shows the identical +1.13mm looser grip, which rules out noise (a truly steady post-close grip should show the same delta across all three, and it does).
2. **The more interesting signal is the missing tail.** cho has episodes that close down to **10.3mm** — a genuinely tight, committed grip. jll's tightest episode only reaches **24.5mm**, more than double that. jll never produces the very-firm grasps cho sometimes does; its distribution is narrower and shifted right. That's a meaningfully different distribution shape, not just a small mean bump.

So: jll's grasps are systematically a bit more gentle, and specifically **lack the tight-grip tail** that cho has. That's plausibly connected to the same underlying cause as the episode-length finding from before — if today's collector lost some early-stop-on-firm-contact behavior, it could simultaneously explain both "always runs the full 209-step FSM" and "never achieves the very tight grips cho sometimes found," since a force/contact-triggered stopping condition would naturally produce *both* variable episode lengths *and* occasional very-tight closures when the optimizer found a case needing extra squeeze.

run	varies	ckpt 400	ckpt 800	ckpt 1200	ckpt 2000
lzhto	baseline	0.35 / 0.385	★0.71 / 0.745	0.61 / 0.645	0.53 / 0.565
zrdoc	horizon=2	0.425 / 0.44	★0.595 / 0.62	0.475 / 0.52	0.345 / 0.38
xvdur	horizon=8	★0.61 / 0.635	0.49 / 0.585	0.46 / 0.645	0.275 / 0.57
diwuc	denoise=10	0.045 / 0.05	★0.33 / 0.365	0.31 / 0.34	pending
kydpe	denoise=50	0.58 / 0.63	0.64 / 0.685	★0.645 / 0.66	pending
cdlus	cond=1	0.37 / 0.79	★0.59 / 0.75	0.555 / 0.655	pending
prfq	cond=4	0.585 / 0.62	★0.595 / 0.62	pending	pending
nrwts	bignet	★0.59 / 0.695	pending	pending	pending

Inference-time ablation on the ★ checkpoint (success_rate / ever_success_rate):

run	act1	act2	act3	act4	ddim5	ddim10
lzhto (ckpt 800)	0.45/0.525	0.66/0.715	0.74/0.79	—	0.66/0.70	0.695/0.725
zrdoc (ckpt 800)	0.42/0.445	—	—	—	0.545/0.60	0.565/0.595
xvdur (ckpt 400)	0.285/0.33	0.43/0.45	0.52/0.545	0.6/0.64	0.345/0.40	0.48/0.515
denoise10/50, cond1/4	not launched yet					

Overall lzhto's own act3 ablation (0.74/0.79) is still the top result across the whole study by both metrics.

(1) Reproduce ahaxs on existing cho data:

run	recipe	ckpt 200/500	400/800/1000	600/1200/1500	800/1600/2000
yhfwl (Run A, 2000ep, save/500)	doc's literal recipe	—	0.575/0.615 (500)	0.655/0.68 (1000)	0.535/0.575 (1500) → 0.52/0.54 (2000)
pfezd (Run B, 800ep, save/200)	doc's literal recipe	0.265/0.30	0.35/0.41	0.46/0.52 (600)	0.46/0.50 (800)
lzhto (true repro, save/400)	matches ahaxs's actual overrides (doc had save_freq wrong)	—	0.35/0.385	0.61/0.645 (1200)	0.71/0.745 (800) → best is ckpt800, not 2000

None hit the reference 0.76 outright, but lzhto/ckpt800 (0.71) gets closest — and its own act3 inference-time variant (0.74/0.79, from the ablation study) actually matches or exceeds the ahaxs reference. yhfwl and pfezd (the doc's literal, slightly-wrong save_freq recipe) land meaningfully lower, peaking around 0.55–0.66.

(2) Re-collect cho + retrain:

run	ckpt 500	1000	1500	2000
dumtm	0.47/0.53	0.435/0.47	0.405/0.495	0.43/0.46

Clearly worse than any Exp1 variant (peaks ~0.47–0.53) — re-collecting with today's code does not reproduce ahaxs, pointing at collector/data drift as a real factor.

(3) rot6d proprio on cho:

run	ckpt 500/400	1000/800	1500/1200	2000
umalx (original, save/500)	0.56/0.575	0.50/0.52	0.47/0.49	0.43/0.445
qsemw (Exp3B, save/400)	0.61/0.645 (400)	0.44/0.49 (800)	0.50/0.545 (1200)	0.455/0.485

rot6d peaks lower than quat's lzhto (0.61 vs 0.71) and, unlike quat, gets worse with more training past its early peak — rot6d proprio looks like a net negative vs quat on this task.

(4) Soft physics, otherwise cho:

run	ckpt 400	800
gordv	0.125/0.59	pending

Big split between the two metrics here — the policy frequently gets the object up (ever_success 0.59) but rarely holds it stably at episode end (success_rate 0.125), consistent with soft-body grasp instability rather than a reach/grasp failure. Checkpoint 800 is still evaluating.

Overall answer to the doc's question: quat (lzhto) is the best of the four axes tested, re-collection (dumtm) and soft physics (gordv's hold-stability) look like the biggest regressions, rot6d (qsemw) is a moderate regression, and none of the doc's literal-recipe reproductions (yhfwl/pfezd) matched ahaxs until the save_freq bug was caught and fixed in lzhto.

Checked that
/home/kei/kei/gentle_manip/logs/dppo/dppo-pretrain/single_lift_mushroom_rigid/cho/lzhto/checkpoint/state_800.pt is okish in real

Bignet (nrwts) exhibits more OOD.

2026.08.18

It is good that we found how to make fem-based policies perform good in sim. After testing in real-world, I found ahaxs is still far more performant. So I decided to pull back a bit and start adding incremental changes on ahaxs (and cho, the dataset it used), to see how do we get a policy that works nice in real-world while trying to have what we want to have (FEM-based synthesis, soft sim, flip DR, etc.)

1. 26-08-18-cak is the version of cho with FEM-based synthesis. Rigid sim, quat.

Also decided to test different training parameters like DDIM steps, pred. horizon, act horizon, cond steps.

Both ablations are running, check them tomorrow and record results properly.

The very high performance we get for ahaxs might be a luck and very much dependent on which ckpt to evaluate. This means the problem might be in learning, because the demo is pretty good already.

Remember, in-sim performance just need to be good enough (like 0.7+), and what we really aim for is a good policy that also do good in real-world. (FEM synth policies already were able to achieve 0.81+, just don't do well in real) What we need to find out is that why ahaxs performed ok in real but the other high performance policies can't, maybe is observation space problem.

- Slightly bigger network helps
 - More data helps
 - Extra squeezing and faster grasp definitely help
- Tested deployment in real, seems policies trained with faster moving demos induce less jerks.
 - Also launched data collection with tighter squeeze and faster grasp phase, scheduled a training and evaluation too, running locally. Should also have some results by tonight.
 - Hypothesis is it will at least perform better than the previous version of v3
 - Data collection success rate went up to ~95%
 - Now best ever_success is 0.81 at 400 epochs of (bwvei)
 - Also collecting 400 extra episodes on top of btd and training a version of (zbqal, the rot6d v3) with combined ~1000 episodes for 600 epochs.(brwkp) This is to see if more dataset would help
 - Hypothesis is no, the behavior problem can't be resolved with more data.
 - Best ever_success is 0.555 at 300 epochs, seems improved a lot over 650 episodes data (previously 0.2 at best)
 - Since training a larger model helped, now we are also training a larger model on 1k data (baeij for x3.5 and lpmo for x7)

- Hypothesis is with larger network and more data, the performance should go up
- baeji has best of 0.425 at epoch 200, no better than the smaller network (brwkp)
- lpmo has best of 0.41 at epoch 200, no better than the smaller network (brwkp)
- Also since larger model helped, we are training an even larger one on the original 650 data. (ecwdp)
 - Hypothesis is that the performance would saturate with even larger network
 - Best is 0.43 at epoch 300, no significant difference than x3.5 version (yubpc)
- Also, we are training on the new (hwo) dataset with harder grasp with bigger net x3.5 (vpstw)
 - Hypothesis is that since performance was better with larger model for old v3 data, this should be even better.
 - 0.815 ever_success at 200, actually almost the same as (bwvei), just came earlier
- Also launched a data collection on cluster using n_grasp_steps to be 21 instead of 30 of hwo, with 3.5mm excess grasp width. This is to find out if faster grasping would mitigate the premature lift. Bignet setup is used. (dppo/dppo-pretrain/single_lift_mushroom_soft_abs_pcd_rot6d_ng21/ctjdi)
 - At best 0.74 at epoch 100, so faster grasp can mitigate it.
- Observing that a main failure mode is during the slow gripper closing, the grasp pose drifts off and the arm starts to lift prematurely. Trying to add an anti-jerk mechanism (smooth alpha) during inference time to see if that helps a bit. Otherwise rely on faster grasps in demo or some other mechanism (like adding small noise during demo grasp so the policy knows a small drift off is ok)
 - On 150 of lpmo, the success rate went from 0.385 to 0.425, not significant.
 - On 200 of yubpc, the success rate went from 0.42 to 0.385, not significant but dropped.
- Trying alpha=0.5 and alpha=0.7 on lpmo ckpt 150
 - Hypothesis is wouldn't help a lot, anyways tuning this is too hacky
 - 0.5: 0.325
 - 0.7: 0.37

2026.08.17

- All runs with v3 synthesis performed bad, /nobackup/proj/disk/softenable-codesign26/personal/ikemura/gentle_manip/logs/dppo/dppo-pretrain/single_lift_mushroom_soft_abs_pcd/arjou/eval/state_400_eval has 0.245, highcam is worse, best 0.1 at 400, highcam_rot6d has 0.2 at 400, rot6d has best 0.2 at 800.
- With hp80, highcam_rot6d has best ever success of 0.3 at 800
- While v2 has ever success with original of 0.6 at 200, rot6d 0.8 at 200, highcam 0.6 at 400, highcam_rot6d 0.4 at 400
- So comparing rot6d with v2 and v3 synth, the former can achieve a best success rate of 0.8 while the latter can only go 0.2. I need to analyze how demo is different so that the training performance can differ this much

- Grasp pose distribution on pitch for v3 has concentration at 0
- Grasp width for v3 is more subtle
- Another hypothesis is that the grasp speed may help. Because longer grasp means larger room for the grasp pose to drift off and go OOD.
- Actually, comparing moxzh having 0.31 success rate, the distribution of grasp pose might not be the reason the performance is poor.
- Also launched x3.5 times larger network training on (btd) (dppo/dppo-pretrain/single_lift_mushroom_soft_abs_pcd_rot6d/yubpc), should have first few results by 18:00
 - Hypothesis is no, it may just overfit
 - Turns out at epoch 200, ever success is 0.42, improved over the original network size. Tho I should be careful that this may be training variance.
- Also launched data collection with tighter squeeze and faster grasp phase, scheduled a training and evaluation too, running locally. Should also have some results by tonight.
 - Hypothesis is it will at least perform better than the previous version of v3
 - Data collection success rate went up to ~95%
 - Now best ever success is 0.81 at 400 epochs of (bwvei)
- So all in all, high cam and/or rot6d were not the main problem, the problem still lies in the demo behavior (or learning parameters). Good that we ruled out some factors.
- Also collecting 400 extra episodes on top of btd and training a version of (zbqal, the rot6d v3) with combined ~1000 episodes for 600 epochs.(brwkp) This is to see if more dataset would help
 - Hypothesis is no, the behavior problem can't be resolved with more data.
- Since training a larger model helped, now we are also training a larger model on 1k data (baeij for x3.5 and lpmo for x7)
 - Hypothesis is with larger network and more data, the performance should go up
- Also since larger model helped, we are training an even larger one on the original 650 data. (ecwdp)
 - Hypothesis is that the performance would saturate with even larger network
- Also, we are training on the new (hwo) dataset with harder grasp with bigger net x3.5 (vpstw)
 - Hypothesis is that since performance was better with larger model for old v3 data, this should be even better.
- Also launched a data collection on cluster using n_grasp_steps to be 21 instead of 30 of hwo, with 3.5mm excess grasp width. This is to find out if faster grasping would mitigate the premature lift. Bignet setup is used.
- Observing that a main failure mode is during the slow gripper closing, the grasp pose drifts off and the arm starts to lift prematurely. Trying to add an anti-jerk mechanism (smooth alpha) during inference time to see if that helps a bit. Otherwise rely on faster grasps in demo or some other mechanism (like adding small noise during demo grasp so the policy knows a small drift off is ok)

2026.08.16

- One observation of OOD is that the gripper closes prematurely, maybe this led to the policy confusing what to do, probe this with fixing gripper width during the first few steps.
 - If this is the case, we can add noise during data collection, or random initial gripper width.
- Trained with highcam rot6d (rla), performance is not satisfactory (qiaal). Not sure why, the distribution of grasp pose is now pretty ok. Possible reasons
 - High cam
 - rot6d
 - some unknown bugs
 - Still the demo quality
 - Also need to test if with v2 synthesis and high cam and/or rot6d the performance is ok.
 - ckpt 200 is doing ok (at least the behavior is fine), 400 and later ckpt get more and more degenerate, looks like OOD during execution
- Collecting the following data
 - v3 synth
 - v3 synth + high cam
 - v3 synth + rot6d
 - v2 synth
 - v2 synth + high cam
 - v2 synth + rot6d
 - v2 synth + high cam + rot6d
 - These should help us find out which component is messing things up, whether it is the synthesis or high cam or rot6d (or combination). At least now from rla qiaal we know v3 synth + high cam + rot6d is not working great
 - Some training results (evaluation videos) should be ready tonight, we can see what goes next
 - One hypothesis is that rot6d has more dimensions so its easier to go OOD. In that case we need to think about how to make the policy more robust to OOD. Maybe add perturbations during demo collection
- Claude was mentioning some eval bugs about EMA, I don't think that is such a big issue.

Updated attempts as of 08.14

Setup:

1. An external camera providing point cloud, facing directly to the robot arm, the camera z-axis is normal to the y-z plane of the robot frame. But this is planned to be changed, we would like to test a higher up camera with 45 or so degrees pointing towards the table, so occlusion is less.
2. No eye-in-hand because point cloud merge was noisy with an external camera, and in many cases objects are out of view initially in eye-in-hand.
3. Point cloud processing

- a. Workspace crop, table point clouds are removed
 - b. FPS as DP3 did, to 1024 points
 - c. Voxel-based filtering, works quite nice in terms of outlier removal
 - d. Per-point Gaussian noise applied in simulation
4. Other in-sim augmentation
 - a. Quaternion is perturbed a bit to mimic real-world noise (very small noise tho, works)
5. In-sim randomization as of now
 - a. Object initial position and orientation, added 25% chance of flipped up-side-down
 - b. Object size and shape (shape variation means randomly bending the object in one of the x-y-z axis by small amount)
 - c. Object stiffness (young's modulus and poisson ratio randomization, mild)
6. For the test object, we currently use mushrooms. Mushrooms bought in supermarkets come with varying sizes and shapes, which poses interesting challenges for the policy. Later we plan to test on different objects with a single policy
7. Sim2real gap evaluation has been done, we did our best to minimize the gap. We checked for point cloud similarity and action following performance, the gap is very small. However, later policy running using absolute actions show larger gap, the gap is mainly in proprioception, we would tune it further, maybe also consider 6DoF rotation representation.
8. For simulation, we use Genesis-world, with MPM-based deformable body simulation where we have access to object internal stress, which we use for measuring gentleness. (lower stress the better)

Attempted methods:

1. Collect 650 demos using a SDF-based grasp synthesis (what we call v2), no gentleness considerations other than having a penetration cost between the finger and object. Trained a absolute-action policy with DP3 on this dataset, in-sim success rate can reach to 88%. Tho real-world performance is much worse. After sim2real gap analysis, proprioception was the main gap, waiting to test different fixes.
2. Meanwhile, a FEM-based v3 grasp synthesis is developed, which truly considers gentleness via stress simulation. Collected 650 demos with this but in-sim performance was bad (<30%), the reason is probably poor diversity of grasp poses, investigating now.

What we are thinking as the updated pipeline:

Step 1: Grasp synthesis-based demo generation in sim (with soft sim for correctness). We hope we can inject fail recovery/retry behavior and robustness somehow in the demo generation process, tho not implemented yet

Step 2: Train a DP3 with the demos (optionally co-train with a few real-world demo)

Step 3: Deploy in real world

No IsaacSim consideration as it requires another round of sim2real tuning and we have limited time.

Development plan:

- ☒ ~~Check DP3 can learn diverse grasp pose~~
- ☒ ~~Develop FEM based grasp synthesis~~
- ☐ Train a good policy (in terms of sim performance) with FEM-based grasp synthesis
- ☐ Close the sim2real gap enough so the policy doesn't perform so worse compared to in-sim
- ☐ Add robustness to demo collection
- ☐ Train for a policy that handles multiple types of object
- ☐

Other decisions:

- We still don't consider eye-in-hand yet, unless occlusion becomes a severe issue.
- We do not consider DemoGen style demonstration augmentation, because grasp-synthesis based demo generation is already automatic and can scale
- Consider learned point cloud augmentation model later, not a high priority task now

Restate todo summarized a few days earlier

1. Test different sim2real approaches, focus on closing the proprioception gap
 - a. Imagined points
 - b. Distance dependent Gaussian noise
 - i. Tho some works show this is not useful
 - c. Camera pose randomization
 - d. 6D rotation representation so there won't be spikes
 - e. Some others recorded in my notes
2. Tackle the jerkiness
 - a. Action chunk post-processing
 - b. Check if the current underlying implementation is delta-based
 - c. Research for methods to make the action smooth
 - d. Add regularization in training
 - e. As a last resort, tune the filter
3. Finish up the grasp quality metric and update the grasp synthesis with it
 - a. Test grasping at the stem and cap
4. Inject robustness to the demonstrations
 - a. Maybe if the mushroom didn't come up after 1cm or so, reopen the gripper, replan and execute, loop until success (with max attempts, maybe like 3)
 - b. We may deliberately make the policy fail, or just solve the currently solved cases with this (which should also make the policy close to 100% success)
 - c. For robustness, maybe we can teleport the object from grasped to mid-air, then replan. That replan is only doable because simulation allows us to plan while the "time" is frozen
5. Train a multi-object grasping policy
6. Write up the updated motivation and introduction

7. Implement held-out evaluation set
 - a. For single-object, held-out means on the DR params
8. Record the randomization parameters encountered during demo, to analyze distribution later
9. IsaacSim implementation, to either train with it or as a cross-sim evaluation
10. Prepare different objects
 - a. Mushroom with different stem size and shape
11. Specify exactly the cfg and commands for training and evaluation (maybe data collection too)
12. Improve the grasp synthesis, try to avoid grasping on stems, and nudge success rate to 100%.
13. In the deploy code, skip the first frame since it is usually corrupted.
14. Add initial gripper width randomization.

From

/home/kei/kei/gentle_manip/dataset/real_deploy/ahaxs800_printed_mushrooms/sim2real_data_analysis, it can be seen that proprioception makes greater impact on sim2real gap, though pnt cld is also contributing some but not significantly.

Grasp synth + DP attempt

2026.08.14

- Found the reason why v3 synthesis has concentration on certain yaw angles: there were 6 fixed yaw seeds, and the bands match them exactly. So it was the yaw seed problem. Pitch had seed angle 0 and also an align cost that encouraged pitch to stay near 0.

2026.08.12

- Trained a policy with v3 grasp synthesis dataset (jcw) (moxzh), success is pretty low somehow, need to dig-in why (best is 0.31)
 - A potential reason is too many occluded side grasp, leading to poor learnability. This might be able to be mitigated by (1) avoid occluded grasp (2) add the first frame to the observation. Anyways, the grasp pose distribution has to be analyzed.
 - Analyzed the grasp pose (euler) distribution, indeed compared to cho, the pitch and yaw distribution is very narrow and irregular.
 - Comparing
 /home/kei/kei/gentle_manip/dataset/demos/single_lift_mushroom_rigid/26-07-29-cho/grasp_euler_distribution.png and
 /home/kei/kei/gentle_manip/dataset/demos/single_lift_mushroom_soft/26-08-08-jcw/grasp_euler_distribution.png, where the former is the dataset collected with v2 synthesis, and yield a DP with 80%+ success rate

2026.08.08

- Data collection with the new grasp synthesis can reach 93% success rate. (jcw)
- Maybe we can add more initial poses to let the policy more robust to OOD execution
- The failed cases make me suspect if the yaw speed is not enough or the planning steps wasn't enough
- Robustness is yet to be added

2026.08.07

- Compared v2 (sdf) grasp synthesis and v3 (fem) grasp synthesis, v3 has higher success rate and lower stress profile. See
/home/kei/kei/gentle_manip/logs/scripted_policy/2026-08-07_16-26-20_grasp_synth_fem and
/home/kei/kei/gentle_manip/logs/scripted_policy/2026-08-07_17-58-15_grasp_synth_sdf

2026.08.05

- Implemented empirical FEM-based stress-aware grasp quality metric
- After the first round of optimization, 10 diverse good performance grasps are selected to further optimize for grasp width (single variable optimization)

Proprioception gap needs to be properly solved.

Imagined points are easy to get, just sample mesh points and transform using FK. See [this](#) for Dexpoint.

Two interesting operations used in DexPoint:

1. Distance dependent Gaussian noise
 - a. Note: not applied to imagined points
2. Camera pose with multiplicative noise for frame transformation
 - a. View point randomization, not simulating extrinsic calibration error, just varying view point

Maybe using 6D rotation representation can solve the quaternion spikes in the w component in real world?

Grasp candidate selection works, but are open loop and cannot easily recover from failure. Sensitive to sensor noise is also a potential issue. [\[cite\]](#) (E.g. "S4G (Qin et al., 2020) proposes a single-shot grasp proposal network based on PointNet++ which assigns each point in point cloud a grasp configuration and its quality score. Non-maximum suppression (NMS) and weighted random sampling are applied to the output to select a grasp to be executed.")

2026.08.04

Some todo summary

15. Test different sim2real approaches, focus on closing the proprioception gap
 - a. Imagined points
 - b. Distance dependent Gaussian noise
 - i. Tho some works show this is not useful
 - c. Camera pose randomization
 - d. 6D rotation representation so there won't be spikes
 - e. Some others recorded in my notes
16. Tackle the jerkiness
 - a. Action chunk post-processing
 - b. Check if the current underlying implementation is delta-based
 - c. Research for methods to make the action smooth
 - d. Add regularization in training
 - e. As a last resort, tune the filter
17. Finish up the grasp quality metric and update the grasp synthesis with it
 - a. Test grasping at the stem and cap
18. Inject robustness to the demonstrations
 - a. Maybe if the mushroom didn't come up after 1cm or so, reopen the gripper, replan and execute, loop until success (with max attempts, maybe like 3)
 - b. We may deliberately make the policy fail, or just solve the currently solved cases with this (which should also make the policy close to 100% success)
19. Train a multi-object grasping policy
20. Write up the updated motivation and introduction
21. Implement held-out evaluation set
22. Record the randomization parameters encountered during demo, to analyze distribution later
23. IsaacSim implementation, to either train with it or as a cross-sim evaluation
24. Prepare different objects
 - a. Mushroom with different stem size and shape
25. Specify exactly the cfg and commands for training and evaluation (maybe data collection too)
26. Improve the grasp synthesis, try to avoid grasping on stems, and nudge success rate to 100%.

From

/home/kei/kei/gentle_manip/dataset/real_deploy/ahaxs800_printed_mushrooms/sim2real_data_analysis, it can be seen that proprioception makes greater impact on sim2real gap, though pnt cld is also contributing some but not significantly.

For robustness, maybe we can teleport the object from grasped to mid-air, then replan. That replan is only doable because simulation allows us to plan while the "time" is frozen

In the deploy code, skip the first frame since it is usually corrupted.

Add initial gripper width randomization.

Consider adding a small gripper noise

2026.07.30

/home/kei/kei/gentle_manip/dataset/real_deploy/ahaxs800_3/sim_replay/2026-07-29_21-10-23 shows that the point cloud in sim and real match but sim pnt needed to shift back by 1cm in x direction to match real. This could be calibration error in real. The trajectory match pretty well except for spikes in w component of the rotation.

Can consider removing the cable point cloud, should be easy to do.

In sim we also need some ways to regularize jittering with absolute control, capping relative motion

Today I'd like to

1. Collect some real-world demo with absolute control
2. Print the mushroom and
 - a. evaluate the sim policy
 - b. Collect some real world dataset for analysis

To robustify the policy we could consider

- Random drop out

Point cloud distribution may also matter, like number of pnts on arm vs on obj

Tested ahaxs again, now with both real and printed mushroom

1. Performance improved a lot with some point cloud shifting, also might be the case that the mushroom I used today is pretty similar to those in sim
2. Using sim mushroom subjectively performs slightly better
3. Found that when the gripper tilts forward and is at the front edge of the workspace, link 7 gets invisible from the depth due to too short range, maybe we need to shift the workspace backward a bit (or tune camera internal parameters)

One sim2real gap could be the point cloud in real is shifted, so it doesn't align with the proprioceptive information seen during training

2026.07.29

Collected 650 demo with improved grasp synthesis (now ~85% success rate, data collection takes 80 minutes) (cho), named cho, try to train a DP3 policy and see how it goes.

- Training with ahaxs, 2000 epochs
 - The performance looked good visually but somehow success rate is very low, turns out it is lifting out of the height band for success. Drifting after lift seems a

problem. We can (1) accept a higher lift band (2) append more stop frames at the end

- After increasing the success band a bit, the ever success rate reached 0.815 at ckpt 800 evaluated locally (roughly 732K iterations)
- Deployed the 800th ckpt in real with action smoothing 0.86 and step cap 0.01, success rate pretty low, see [video](#). The main problem is faulty approach sometimes, maybe its the sim2real gap
- Also I noticed that when the arm move to the right, part of the gripper gets out of view from depth camera. Maybe we need to move the workspace more to the left so the center aligns with the camera.
- Training Unet version with ruyhb, with 1000 epochs
 - Not as good

The first thing I would do is to print the mushroom models we used in sim to

1. Visually compare how different they are from real mushrooms
2. Place the printed mushroom at a known location, compare the point cloud with the same setup in sim, see where exactly the gap happens

Some other way around is

1. Co-train with real-world demo
2. Other sim2real robustify method
 - a. Adding low pass filter to real quaternion
3. Try using 6d rotation representation instead of quaternion?
4. Simulate edge warping like the L515
5. Get the robot arm point cld discrepancy solved first

2026.07.28

To summarize the result so far:

1. Absolute action control rocks, best is so far 0.52, perfect approach, failure modes are missed grasp or object rolled. We will base future development on this
2. fjiyis running to see if longer training would give higher result (and also check reproducibility of hqbve)
 - a. Best result is 0.365 at ckpt 3500, seems the 0.52 was an outlier. But still absolute action control is much better. The major failure mode is grasp width

Todo:

1. Test the best-so-far in real
2. Need a better grasp synthesis and control mechanism, aim for
 - b. High success rate, ideally 100%
 - i. **Retry grasping harder using force feedback**

- c. Minor but maybe use other algorithms than CMA-ES or finetune CMA-ES parameters
 - d. As gentle as possible, can use force feedback
 - e. Prefer per-env control, instead of all envs synching in phases, though not sure if this would create learning challenge
 - 3. Add backend guards to prevent abrupt motions
 - 4. Inject robustness and retry
 - a. To improve the success rate I have two ideas (1) when the target grasp width is reached at grasp phase, if the object's force value is less than 1N for a particular env, close 2.5 mm more on that env (other env can start lifting), do this only once (2) If in lift phase, like after the gripper goes up to 1.0cm, the object did not come up, replan or open gripper a bit and goes back to the grasp pose and regrasp. We can deliberately perturb the grasp width to make the grasp fail by some probability $\epsilon=0.05$ for example and trigger retry.
 - b. Both are good and now genuinely implementable thanks to the per-env decoupling. Idea 1 is the simpler, safer one — it directly closes the loop on the `priv_contact_force` signal we already validated, is bounded to a single top-up so there's no risk of over-squeeze, and fixes a real failure mode (CMA-ES's cost proxy undershooting actual grip firmness). I'd implement that first.
 - c. Idea 2 is more valuable long-term — it's the difference between a demonstrator that only shows "grasp works" and one that shows "grasp failed, here's how to recover," which matters a lot for what the learned policy generalizes to at deployment. Two things worth deciding before I build it: (a) on detected slip, do you want cheap regrasp-in-place (reopen + re-close at the same CMA-ES pose — object likely hasn't moved much) as the first attempt, only falling back to a full CMA-ES replan if that fails too (replanning per-env mid-batch is now feasible since a slow env no longer blocks the others, but it's still real wall-clock cost you'd be paying per retry); and (b) a hard retry cap (e.g. 1–2 attempts) so a fundamentally bad env can't stall the whole batch or produce a runaway-length episode.
 - d. One shared risk: the epsilon-perturbation should mimic a realistic failure (under-closing the width, as you suggested) rather than something contrived, or the recovery behavior the policy learns won't transfer to genuine real slip. Want me to start with idea 1?
 - 5. Consider
 - a. Now to solve for accurate grasp width, we may try
 - i. Deal with size ambiguity from single view
 - 1. To deal with size ambiguity from only one view, we have a few options to solve
 - a. Use two views, this might face more severe sim2real gap
 - b. Restrict the grasp poses to be roughly facing front, but this may be too limiting, if the only possible grasp pose is not facing front, we are done
- Regress grasp width as an auxiliary objective

1. More data and bigger network
2. Finetune only the grasp width action axis with force/stress penalty, we can justify this because the pose has been properly handled
 - a. Can test asymmetric actor-critic
5. After some execution the sim gets slow, I remember encountering this before but forgot what's the fix (self-collision check?)
6. Can try more types of augmentation/randomization to make sim2real as robust as possible, compute is not a big problem
7. If point cloud only hardly works, we have to try out image-based, which means we need IsaacLab

1. Try to train on fixed home demo (jud) and evaluate on fixed home demo, with 300+ demo, want to see what is the ceiling of performance we can get.
 - a. The best performance is obtained by (klrhd) at epoch 1000 (1500 drops), success rate 0.62.
 - b. No or lower initial pose variation indeed gives better performance, but we don't want to reduce this, 4cm box is already pretty tight. But this may suggest that we can get away with more demonstration data
 - c. One thing that somewhat concerns me is that this 0.62 success rate is awfully similar to the success rate of the scripted policy (0.6-0.7), is the data collection process biasing the data? The sma dataset had 0.76 success rate (no object shape/size randomization), which also trained to produce a best performance of 0.77. I kind of feel like the demonstrator is the bottleneck
2. Train on a state-based demo (pvm), though it can also be used for point cloud training as the data is stored.
 - a. The best performance is obtained by (wayhx) at ckpt 1500, success rate 0.35, way higher than previous 0.12 with point cloud input, though still not satisfactory.
 - b. And now most failures are due to wrong grasp width, the approach phase is mostly correct.
 - c. Seems 0.35 is the best we can get with this setup, another run (mldib 2026-07-28_16-41-16_42) gets 0.3 at 4000 epochs
 - d. State-based does make the policy perform stronger, but only marginally
3. Implemented absolute action control and train on a point-cloud-based, full DR demo (rpk), 300+ demo.
 - a. Some primary results (hqbve) show the approach gets much more accurate even on full DR. We confirm that the grasp poses are very well executed, this settles that absolute actions are much more superior for accurate grasp pose reaching than relative actions, the remaining question is how to grasp well with appropriate grasp width
 - b. Best result so far achieved by (hqbve) at 2000 ckpt, 0.52, very nice. Need to check for reproducibility tho. Now running a longer training with 3500 epochs. With 2000 epochs we are hitting 1M iterations, 3500 is about 1.5M+
 - c. Somehow the loss curve is also nicer (hqbve)

- d. The robot initially moves very fast, and during execution there could also be jitters. I'd say do something on the backend side to prevent drastic and/or high frequency motion
 - e. If this turns out working, we can (1) test this in real world (2) think about injecting robustness or lean more toward soft body refinement
 - f. Some other potential improvements are
 - i. The KL pose reconstruction
 - ii. Faster architecture
 - iii. Or better architecture
 - iv. Or focus on getting the full pipeline works, and test on other objects
- Ruiyu mostly use absolute actions
 - Absolute actions give less workspace generalization but is more robust to error accumulation
 - Thus absolute actions should be more robust in sim2real sense.

This paper is directly relevant: DextrAH-RGB: Visuomotor Policies to Grasp Anything with Dexterous Hands

OmniReset uses relative cartesian space control.

OmniReset talks about seed selection. I like honesty.

In OmniReset, despite many efforts, the distilled policy can only achieve ~50% success rate in simulation, while a state-based RL policy can reach almost 100%. This suggests that distillation is not perfect.

In OmniReset, they generated 80,000 episodes of expert trajectories to distill a student policy, with a simple MLP architecture plus a pre-trained ResNet encoder. The training took 2 days on one H200. "The policy consists of an ImageNet-pretrained ResNet18 encoder followed by a 4-layer MLP with 512 hidden units". What a simple network architecture. Also "Following DextrAH-RGB (Singh et al., 2025), we find that combining pose reconstruction with KL matching to the expert's action distribution leads to the best distillation and transfer performance. In particular, our most performant policies are trained with a KL loss and deployed only using the mean action. **Without pose reconstruction, the RGB policy sometimes fails to reliably localize and grasp objects.** We also find that end-to-end training, with gradients propagated through the visual encoder, is critical; freezing the encoder significantly degrades performance."

They trained for 350K gradient updates, with a batch size of 64, it means the policy is trained with about 22M transitions. Assuming 80K rollouts, each with 150 steps, then this is roughly training for 2 epochs. If we are using 300 demos or 62,073 frames, and batch size of 128, we are getting 485 iterations per epoch, meaning ~750 epochs would be reaching the same number of iterations as OmniReset.

2026.07.27 - 07.28

Trying to train now with full DR-based demo collection (hfx). Collected 330 demos with full DR.

1. Trained a basic DP3

- a. dppo/dppo-pretrain/single_lift_mushroom_rigid/hfx/ozrtb
- b. The best performance is at 1000 epochs, with full DR eval, success rate is 0.12, pretty low. Diverse grasp behavior is shown but high failure rate
- c. With fixed home at nominal position, other full DR eval, success rate climbs to 0.52, clearly the policy is sensitive to initial position
- d. Diagnostics:
 - i. Confirmed that the demo does cover diverse samples from the home position randomization box
 - ii. The success distribution of initial position can be seen here
 1. logs/dppo/dppo-pretrain/single_lift_mushroom_rigid/hfx/ozrtb/eval/state_1000_eval_rigid_grp4/gripper_offset_success_rotation.gif
 2. It is clear that there are success cases spread to the entire initial position randomization box
 - iii. Maybe the variation in grasp pose + pntcld + full DR is just too difficult to learn?
 1. Try state-based
 - iv. Maybe the action representation is unsuitable?
 1. Try absolute action
 - v. Maybe the network size is insufficient?
 1. Try bigger model
 - vi. Maybe the number of samples is not enough for severe randomization like what we have?
 1. Idk about this, we used the same setup but 50 demos with top down grasp, that worked. This may suggest that it is the behavior distribution problem
 - vii. How come OmniReset can train a model with 8k demos? What architecture do they use?
 - viii. Maybe something else is needed for adapting to this diverse behavior?
 - ix. Would the success rate be high given any fixed home position? So it isn't the varying home position a problem, it is varying home position + other DR a problem. If you fix a home position, the success rate would still be high with other DR, this need testing
 1. With offset fixed at [0.01, -0.01, -0.005], success rate of the same ckpt was 0.255, dropped compared to fixed at home case
 2. With offset chosen as the first failed case for full DR, success rate dropped to 0.01
 - x. In the failed cases (of full DR), how many are approach failures and how many are grasp failures?

1. Although not complete [here](#), but majority is approach location being wrong
- xi. Architecture problem?
 1. Trained on Unet 1250 (ktvfb) epochs and 1500 epochs (jqiej)
 - a. The success rate is even lower with full DR, was 0.07 by ckpt 500 of jqiej
 - b. With fixed home at nominal, highest success rate achieved by the same ckpt with 0.585, also lifted significantly
 - c. This just shows Unet might overfit harder
- xii. All-in-all, the diagnostic is that the policy does generate diverse grasp behavior, but mainly fail for a good approach given heavy DR. Given that the policy was able to succeed with the same setting with top-down grasp, we suspect is the behavior complexity that causes the problem.
- e. We might also need to tune for learning rate?

2026.07.25 - 07.26

Collected 320 demos using first version grasp synthesis on rigid object grasping. Locally trained 2000 epochs (which was enough before).

Run: dppo/dppo-pretrain/single_lift_mushroom_rigid_pcd/qhycy

The result shows:

1. Zero success rate at 1k epochs
 - a. Mainly because of point 3 below
2. The grasp behavior is almost the same regardless of how objects are placed
 - a. One reason could be the point cloud is too small to tell the difference
3. After approaching, the gripper does not close
 - a. This might be due to having too many zero-action frames between end of approach and start of grasp

Problem 3 diagnostics:

1. Turns out the grasp action is only one frame, there rest are zero-action, this can be seen from the zero-action histogram. That action is saturated at -1, can be seen from the action distribution
2. Also too many consecutive zero-actions.
3. Also from the action distribution plot we found that we were not using the fastrot config. Because Rotation axes (d_roll, d_pitch, d_yaw) are saturating at ± 1.0 . This might be contributing to the fact that the learned policy doesn't behave right

Changes to grasp synthesis collector

1. Now grasp is a gradual process
2. Grasp synth allows wider orientation coverage
3. With fast_rot action cfg and less steps

4. Settling is taken care of in env
5. Parallel CMA-ES
6. Wiring the env setup with proper experiment loading

Remaining grasp synthetic issues

1. After some execution the sim gets slow, I remember encountering this before but forgot what's the fix (self-collision check?)
2. Mushrooms are smaller than we
3. High grasp failure, not sure why
 - a. Scene settle was a problem

In the mah dataset, we confirmed a few things

1. Less all-zero-actions
2. Action magnitude distribution is fine
3. No single action axis saturates

Would obs history actually be bad for deciding when to stop closing grippers?

Orientation constancy diagnostics:

1. After fixing zero-action frames and initial settling frames
2. Before scaling anything, overfit a tiny dataset: two object poses (say upright and 90° side-lying), each paired with its distinct correct grasp, dozens of demos each. Then ask whether the trained policy produces the two different grasps conditioned on which pose it sees.
 - a. Using a 50 demo dataset with confirmed pose diversity, quick test (mah). The general idea is if the policy can't behave basic functions when the network already overfit, there are fundamental problems.
 - i. dppo/dppo-pretrain/single_lift_mushroom_rigid/mah/orzqp
 - ii. Trained, confirmed that different grasp behavior CAN be learned, its just the success rate is quite low, grasp position is off most of the time. Success rate is about 20%
 - iii. One possibility is too few data, the other is failed demos are filtered but they actually comes in the eval process (i.e. bias in demo) (see point (b) below)
 - iv. Network capacity
 - v. I think the val loss is no longer going down so it fitted quite ok
 - vi. at least improved over v1
 - vii. A major suspect is covariate shift, which maybe more sever for delta-action based control, for this we might need to try absolute pose control. Or RL for rescue. But I think the base performance can't be too bad
 - viii. Another possibility is point cloud observation, either too ambiguous or too hard for network to learn on. To probe this we can train a state-based policy

- ix. Correct eval result: 0.61 at 1500 epochs, 0.49 at final epoch. Dispute no object size/shape randomization at data collection. Clear overfitting at 3000 epochs
- b. Using 200 demos did not produce any improvement (though pose diversity is still there)
 - i. dppo/dppo-pretrain/single_lift_mushroom_rigid/sma/apioc
 - ii. Success rate 0.78 at 2000 epochs, significantly better than 50 demo cases. Dispute no object size/shape randomization at data collection
- c. Run UNet-head policy with the same 200 demos in (b)
 - i. dppo/dppo-pretrain/single_lift_mushroom_rigid/sma/ytmax
 - ii. Better val loss and train loss
 - iii. But poor success rate (best 0.02), seems the policy ignores point cloud completely, is this something we call the copy cat problem?
 - 1. Well, the action does change w.r.t. point cloud difference, so its not completely ignoring the point cloud
 - iv. According to Claude: the UNet fails because it fit the demos more sharply — lower BC loss = a sharper, higher-Lipschitz function = worse extrapolation to the states rollout actually visits. Classic BC compounding-error / covariate shift, and the more expressive head is more susceptible.
 - v. Correct eval result: 0.77 at 2000 epochs, no statistical difference than the MLP version, but is running much slower
 - vi. From the val curve we can see that the overfitting is more severe
- d. Try to train a state-based policy
 - i. Learned 6D rotation representation and we are using it for obj pose
- 3. Verify the diversity is actually in your data's action space, not just in object pose.
 - a. Yea, there are. We checked with analysis scripts.
- 4. Could be the copycat problem, which means expert actions are strongly correlated over time, the imitator learns to cheat by predicting the expert's previous action rather than the next one. Grasp trajectories are exactly that — smooth, temporally correlated descents — so if your observation includes end-effector-pose history or previous actions, the policy can regenerate the motion from its own momentum and ignore the point cloud entirely
 - a. Not quite getting this
 - b. Ablate proprioception / shrink observation history to rule out copycat. Drop the end-effector-pose and previous-action inputs (or set observation horizon to 1) and retrain. If the policy suddenly starts conditioning on the object, it was cheating off its own motion history, and you fix it by removing that shortcut or de-correlating it.
- 5. Check your action frame — this connects to the absolute-vs-relative point from earlier. If actions are delta-pose in the end-effector frame, "approach along the grasp axis and close" looks nearly identical across object poses in action space even when the world-frame grasps differ wildly. The Diffusion Policy paper found absolute/positional

control generally outperforms relative, and this is partly why. Try predicting the grasp in an absolute or object-relative frame and see if the diversity reappears.

6. Probe whether the encoder even sees pose. Freeze the DP3 point-cloud encoder and train a tiny linear head on its features to predict object orientation.
 - a. If no, one possibility is that the object point cloud is too tiny w.r.t. the scene point cloud
7. Check mode balance and coverage. If most demos are one canonical grasp and side grasps are a small minority, diffusion will sample the dominant mode; balance the dataset across poses. And MimicGen explicitly warns that biased scene coverage under-samples certain configurations unless the reset distribution is carefully engineered — sparse sampling of the orientation space looks like collapse but is really under-coverage

Found a crucial bug: the demonstration never had domain randomization other than obj position. So size, bend, and robot initial position are never randomized, this is why the evaluation results were so bad for 50-demo, 200-demo, and 200-demo-unet runs.

Tested the policy that trained on 200 demos for diverse grasp pose (on rigid objects) in real

- dppo/dppo-pretrain/single_lift_mushroom_rigid/sma/apioc/checkpoint/state_2000.pt
- [Does generate diverse grasp pose, pretty nice.](#)
- The failure mode is mainly insufficient approach and too much grasping (reasonable since at some pose the object is occluded, plus the objects used in sim are smaller than what the real world has)
- The core message is that training on diverse grasp poses works and can transfer to real world (though the gap in orientation command should be evaluated still)
- Also checked sim2real gaps, not much, though some cases the angular error spikes, I think mainly is the simulation joint gain insufficient, but I'd not change as of now

There is evidence that DP can give different rotation behavior depending on scene configuration, for example in MimicGen and the follow-ups (e.g. SoftMimicGen).

Our attempts as of 07.24

Setup:

9. An external camera providing point cloud, facing directly to the robot arm, the camera z-axis is normal to the y-z plane of the robot frame.
10. No eye-in-hand because point cloud merge was noisy with an external camera, and in many cases objects are out of view initially in eye-in-hand.
11. Point cloud processing
 - a. Workspace crop, table point clouds are removed
 - b. FPS as DP3 did, to 1024 points
 - c. Voxel-based filtering, works quite nice in terms of outlier removal
 - d. Per-point Gaussian noise applied in simulation
12. Other in-sim augmentation

- a. Quaternion is perturbed a bit to mimic real-world noise (very small noise tho, works)
- 13. In-sim randomization as of now
 - a. Object initial position and orientation
 - b. Object size and shape (shape variation means randomly bending the object in one of the x-y-z axis by small amount)
 - c. Object stiffness (young's modulus and poisson ratio randomization, mild)
- 14. For the test object, we currently use mushrooms. Mushrooms bought in supermarkets come with varying sizes and shapes, which poses interesting challenges for the policy.
- 15. Sim2real gap evaluation has been done, we did our best to minimize the gap. We checked for point cloud similarity and action following performance, the gap is very small.
- 16. For simulation, we use Genesis-world, with MPM-based deformable body simulation where we have access to object internal stress, which we use for measuring gentleness. (lower stress the better)

Attempted methods:

- 3. Collect 50 gentle grasp demos in the real world, train a DP3 and test.
 - a. During the demo collection, we use many different mushroom instances to cover shape and size variation. We also put the mushroom in different initial locations
 - b. The demo does not consider grasp orientation yet, we just approach the object and grasp with appropriate width. The reason is to not stress the policy to model different grasp pose yet
 - c. The real world result shows that the policy can approach the object location correctly most of the time, but the major failure mode is inappropriate grasp width, either too hard (so object crushes) or too wide (so object does not come up). Also no re-try behavior is shown. In some rare cases, the mushroom also rolls because the grasp center was not aligned, making the mushroom escape from the grasp.
- 4. Collect 150 demos in simulation with scripted policy, train a DP3 and zero-shot deploy in real.
 - a. The scripted policy does not consider orientation variation, same as our human demo, just descends and grasps. Notably, the grasp is adaptive to object size because we used the object size knowledge to adjust our grasp width. This also gives gentle behavior
 - b. In-sim success rate is 0.82.
 - c. The real world result shows a higher success rate than the first method (real-world DP) we tried. Probably thanks to the higher number of demos. Though the failure mode is the same, missed grasp, crushing the object, and object rolls over but never retry. This is understandable because our demonstration does not consist of retry behavior. The success rate is about 0.6, and among the success cases, 40% were safe, while 60% ended in mushroom bruising or breaking.

5. Fine-tuned the policy in (2) with DPPO, with regular task rewards (dense reward) plus a penalty term based on object internal stress, basically promoted for gentleness.
 - a. In-sim success rate went up from 0.82 to 0.9, real world results are more or less the same.

What we are thinking as the next pipeline:

Step 1: Grasp synthesis-based demo generation in sim (can even use rigid sim for fast generation). We hope we can inject fail recovery/retry behavior and robustness somehow in the demo generation process.

Step 2: Train a DP3 with the demos

Step 3: Optionally finetune with DPPO (diffusion policy policy optimization)

Step 4: Deploy in real world

Concurrently: try IsaacSim because it seems more widely used and maybe faster (actually is faster than genesis-world)

Development plan:

- We first check if a plain DP3 can consume grasp-pose-diverse data by training on diverse grasp poses demo generated on rigid sim (quick) with any simple grasp pose synthesis (suffice as long as the grasp pose is diverse, we don't care about stress-minimization yet). If it can, we proceed with better grasp synthesis, if it can't we need to use alternative architectures. Another potential challenge is that with grasp poses that would block the object (by fingers) during grasp, the policy may fail. A naive fix I can think of is to condition each timestep with initial point cloud where object is fully visible. Here we use genesis as our code it is already developed quite good on it
- Since some evidence shows DP3 saturates around 300 demos, we will go with 300 demos with as much object pose, shape, and size diversity as possible
 - Nope, some other works use 80K demos, 300 is too few for diverse initial conditions. The saturation is given their narrow initial conditions.
- Develop IsaacSim code in the meantime (while the policy trains), maybe we can leverage it for faster development later
- Check if force limiting and current/force feedback works on the gripper. If yes, limit the force and consider force-adaptive control.
- A gentle grasp is not only concerning grasp force/width, but also pose. Clamping at the correct location is as important as how much you clamp.

Now we confirmed that diverse grasp pose can be learned and can be transferred to real, we can now proceed with the following:

- Stress-minimized grasp quality metric
- Inject robustness and retry behaviors
- Test performance in soft simulation
- Optionally finetune in soft simulation

Other decisions:

- We still don't consider eye-in-hand yet, unless occlusion becomes a severe issue.
- We do not consider DemoGen style demonstration augmentation, because grasp-synthesis based demo generation is already automatic and can scale
- Consider learned point cloud augmentation model later, not a high priority task now

Injecting retry and robustness into generated demos (your Q10)

This is the most actionable item, and grasp synthesis in sim makes it clean:

1. **Scripted recovery in the generator.** In sim you have ground truth — detect a failed grasp (object not lifted, gripper closed on empty width, or internal stress exceeding a bruise threshold), then script: back off, re-perceive, re-approach, re-grasp. Record the *whole* trajectory including the failure and recovery. This is the single most direct way to get retry behavior — the policy can only learn retry if retry appears in the data, which is why your current policies never retry.
2. **Deliberately seed failures.** Script a fraction of episodes where the first attempt is designed to miss (offset the grasp center, wrong width), then have the expert recover. This teaches the recovery manifold explicitly rather than hoping it appears.
3. **Perturbation-and-correction (DART/Dagger-style).** Inject noise into the state mid-trajectory, then have the scripted expert correct back to nominal. This populates the off-distribution states around your demos so the closed-loop policy doesn't diverge when it drifts — this is what buys general robustness, not just retry.
4. **RL finetuning you're already doing (DPPO).** RL will *discover* retry on its own — but only if the episode doesn't terminate on first failure and the reward credits a successful regrasp. Check your termination/reward: if the episode ends the moment the first grasp fails, DPPO can never learn to retry. Give partial credit for re-approach and let episodes run long enough for a second attempt.

Regarding what is summarized by Claude on 07.25

“published evidence shows vanilla DP3 degrades sharply as object pose/orientation randomization grows (DemoGen reports DP3 at 16%/13% max/avg success in enlarged-randomization sim tasks and ~11% in the real world when trained on un-augmented data), and it exhibits an explicit "visual mismatch" failure precisely when observed orientation deviates from demonstrated orientation.”

“DemoGen's analysis states the "spatial effective range of visuomotor policies is closely tied to the distribution of object configurations seen in the demonstrations." Un-augmented DP3 shows severe overfitting: 16%/13% (max/avg) in enlarged-randomization sim tasks from a single

source demo, and ~11% real-world average — "blindly replicating the demonstrated trajectory." With DemoGen augmentation, the same DP3 reaches ~88% sim / ~75% real."

I need to check DemoGen results. If training on augmented data gives enough performance boost, what about the evidence that DP3 has poor scaling w.r.t. number of demonstrations? And if this is enough, why consider equivariant architectures?

This evidence can be used as an argument for why we shouldn't rely on real-world demo. And our experiment shows that plain DP3 is also poor for varying object sizes if gentleness is of concern (otherwise we just use binary open/close command with force limit, that way everything can be grasped)

"more simply, solve orientation issue via eye-in-hand + relative/delta action parameterization, which is provably SE(3)-invariant."

"iDP3 (Improved 3D Diffusion Policy) — Ze et al., arXiv:2410.10803. Egocentric 3D representation; removes dependence on precise calibration/segmentation; deployed on humanoids."

Eye-in-hand, when combined with an external camera, creates misalignment during point cloud merge, to me this is suboptimal and increases the complexity. While pure eye-in-hand is also not preferable since the object may initially be out-of-view, how does the original paper deal with this?

"for "delicate" grasping — add force/compliance signals, since vision-only position control is insufficient for fragile objects." - we do not have access to force feedback, does any work solve this? Or if force feedback is somehow available on Xarm7.

"ACRONYM — Eppner, Mousavian, Fox, ICRA 2021 (arXiv:2011.09584; github.com/NVlabs/acronym). "The dataset contains 17.7M parallel-jaw grasps, spanning 8872 objects from 262 different categories, each labeled with the grasp result obtained from a physics simulator" (objects from ShapeNetSem, ~2000 grasps per object, ~59% success labels). Grasp transforms are in the Franka Panda gripper frame. Google

Contact-GraspNet — Sundermeyer et al., ICRA 2021 (arXiv:2103.14127; github.com/NVlabs/contact_graspnet). End-to-end 6-DoF grasp distribution from a single depth/point cloud; reduces the grasp representation to 4-DoF by rooting contacts in observed points; trained on 17M sim grasps.

GraspGen — Murali et al., NVIDIA, 2025 (arXiv:2507.13097; graspgen.github.io; github.com/NVlabs/GraspGen). Diffusion-Transformer 6-DoF grasp generator + on-generator-trained discriminator; 53M+ simulated grasps; zero-shot sim-to-real; multi-gripper.

GraspNet-1Billion (Fang et al., CVPR 2020) and Dex-Net (Mahler et al.) are the other standard grasp-synthesis references”

“Grasp-synthesis datasets (ACRONYM: 17.7M grasps; GraspNet-1B; Dex-Net) and 6-DoF grasp generators (Contact-GraspNet, GraspGen) can seed diverse SE(3) grasp poses; demo-generation systems (MimicGen, DemoGen, CP-Gen, SkillMimicGen) then turn a few source demos into thousands covering pose/orientation variation.”

Does it mean when object orientation changes, the corresponding grasp pose also rotates by the same amount? Or how does it handle orientation change? Certainly a new grasp pose is needed.

For a **rigid** object, yes — grasp poses live in the object frame, so if the object undergoes SE(3) transform T , every valid grasp transforms by the same T . That's exactly the object-centric assumption MimicGen/DemoGen rely on: record the demo segment relative to the object pose, then for a new object pose apply the relative transform to the whole segment (approach + grasp). A "new grasp pose" in the world frame, yes, but it's the *same* grasp in the object frame, rigidly transported.

The catch for you: mushrooms are deformable and shape-varying, so the rigid-transform assumption breaks. Your shape randomization (bending in an axis) and size variation mean the "same grasp in object frame" isn't well-defined across instances — the grasp must adapt to the *actual* geometry, not just be rigidly transported. This is why pure rigid-transform augmentation à la DemoGen is necessary-but-insufficient for your case: you need per-instance, geometry-aware grasp synthesis (a Contact-GraspNet / GraspGen-style generator run on each instance's point cloud), not just SE(3) replays of one canonical grasp. This is arguably the strongest reason in your whole plan to adopt grasp synthesis: it handles shape variation that transform-augmentation cannot.

For all of these, is the dataset used for continuous control? Or when deployed the policy still predicts a single grasp pose and executes in an open-loop manner?

“Equivariant Diffusion Policy — Wang et al., CoRL 2024 Oral (arXiv:2407.01812; equidiff.github.io; github.com/pointW/equidiff). SO(2)-equivariant denoiser; "we evaluate the method empirically on a set of 12 simulation tasks in MimicGen, and show that it obtains a success rate that is, on average, 21.9% higher than the baseline Diffusion Policy" (voxel input, 100 demos, absolute control); benefit strongest in the low-data regime and on "high-equivariance" tasks. The paper theoretically analyzes the SO(2) symmetry of full 6-DoF control. Equidiff

EquiBot — Yang et al., CoRL 2024 (arXiv:2407.01479; equi-bot.github.io). SIM(3)-equivariant (scale + rotation + translation) diffusion; generalizes to novel objects/scenes from ~5 min of human demos.

Spherical Diffusion Policy (SDP) — Zhu, Wang, Walters, Shi, ICML 2025 (arXiv:2507.01723; github.com/amazon-science/Spherical-Diffusion-Policy). Full SE(3)-equivariant via spherical Fourier space; gains over strong baselines on 20 sim + 5 real tasks (single-arm and bimanual). Limitation: position control only.”

For all of these, on what tasks they evaluate where orientation becomes crucial?

Larger 3D policies (transformer / voxel / feature-field):

- 3D Diffuser Actor — Ke, Gkanatsios, Fragkiadaki, 2024 (arXiv:2402.10885; 3d-diffuser-actor.github.io). Diffusion + 3D scene tokens + relative attention; SOTA on RL Bench (+16.3% multi-view, +13.1% single-view absolute gain over prior SOTA), works in the real world from ~15 demos/task, and "is able to solve multimodal tasks in the real world" (stacking blocks/cups). On multi-view RL Bench it reports 81.3% avg across 18 tasks (+18.1% over Act3D). 3d-diffuser-actor
- Act3D — Gervet et al., 2023 (arXiv:2306.17817); RVT / RVT-2 — Goyal et al. (RVT-2 arXiv:2406.08545); PerAct — Shridhar et al. (voxel, language-conditioned); GNFactor; ChainedDiffuser. These are keypose/open-loop or multi-view transformer policies; ranking on RL Bench: 3D Diffuser Actor > Act3D > GNFactor.

These are interesting and we can check their training recipe, as Finn did to learn large-scale training.

DiffuDepGrasp — 2025 (arXiv:2511.12912). RL teacher → diffusion depth-noise model → depth student; zero-shot sim2real; 95.7% average on 12-object grasping.

DiffuDepGrasp(DDG) is structured into four key stages. (A) Teacher Policy Training: An RL-based teacher policy is trained in simulation with privileged state information to generate expert demonstrations. (B) Diffusion Depth Generator: This core module learns to simulate realistic sensor noise. It consists of a Diffusion Depth Module that learns noise patterns from real data, and a Noise Grafting Module that injects these patterns into perfect simulation geometry. (C) Student Policy Distillation: The teacher's knowledge is distilled into a vision-based student policy using our generated high-fidelity depth data. (D) Sim2Real Deployment: The final student policy is deployed zero-shot on the physical robot.

The noise augmentation model is interesting, maybe something useful. Also, it seems they only have 1 external view, how is occlusion handled?

They don't really solve occlusion — single external view means partial point clouds, and they accept that. The trick that makes it work is the privileged-teacher / partial-student distillation: the

RL teacher trains on full privileged state, and the vision student is distilled to act from the partial, noisy single-view depth. Occlusion is handled implicitly by training the student to succeed despite partial observation, not by de-occluding. Their actual contribution is the depth-noise model (learn real sensor noise, graft it onto clean sim depth), which is the part worth stealing for your sim2real — it's a more principled version of your per-point Gaussian noise, since real depth noise is structured and includes dropout/voids that Gaussian doesn't reproduce.

“Diminishing returns from brute-force demos. For DP3 in a full-workspace setting, going 100→150 demos gave +37% but 150→200 only +6% — so simply adding demos to cover SE(3) saturates; smarter coverage or symmetry is needed.”

Isn't this going against DemoGen's argument? If increasing the demo does not help.

- Nvm, this argument comes from DemoGen
- But still, isn't this diminishing the contribution of DemoGen? Or they argue that a better model can consume more data effectively?
- “On the number of demonstrations, while task performance generally improves with an increased number of demonstrations, the effect diminishes beyond a certain threshold. For instance, in the full workspace setting with DP3 as the policy, a 50 demonstrations increase from 100 to 150 enhances the performance by 37 % , but the increase from 150 to 200 only improves the performance by 6 % . This finding also highlights the inherent difficulty in achieving near-perfect success rates in robot learning systems.”

These three questions are really one confusion, and it dissolves once you separate **count** from **coverage**.

- The "150→200 gives only +6%" saturation is about adding demos *at fixed coverage* — more samples from a distribution you already cover densely. That saturates, as you'd expect.
- DemoGen's contribution is **coverage**, not count: it places synthetic demos at object configurations you'll actually encounter at test time, expanding the "spatial effective range" to match the test distribution. The 100→150 jump of +37% was still expanding coverage; the 150→200 of +6% was past saturation. So there's no contradiction — generation helps because it changes *what* distribution you sample, not merely *how many*.
- So "use generation" is not "more of the same demos." Adding 50 more top-down mushroom grasps at poses you already cover won't help; generating grasps at *poses/geometries you don't yet cover* will. That's the whole distinction.

“Rotation representation: Use the 6D continuous rotation representation (Zhou et al., CVPR 2019, arXiv:1812.07035) rather than quaternions or Euler angles. Quaternions/Euler are discontinuous and "difficult for neural networks to learn"; all ≤ 4 -D representations of SO(3) are discontinuous. 6D (Gram-Schmidt) is continuous, singularity-free, and empirically yields lower angular error and smoother training — important when regressing varying grasp orientations. ”

Will take this into account.

“if dithering/mode-collapse appears, consider conditioning on a selected grasp (affordance/goal conditioning, e.g., AffordDP).”

Will take this into account. Also initial point cloud maybe, to handle occlusion.

“Number of demos: DP3 handles many sim tasks with 10 demos and real tasks with ~40 demos/task for modest variation. But for wide orientation variation, demo counts saturate (150→200 gave only +6%). Equivariant policies cut data needs substantially (EquiBot from ~5 min of demos; EDP strongest at 100–200 demos). Net guidance: don't rely on demo count alone to cover $SE(3)$; use generation + symmetry.”

I don't get it, wouldn't “use generation” basically mean increasing demo count?

“Point-cloud augmentation: Standard and important — point jittering, random dropout, random cropping, farthest-point-sampling downsampling, and voxel downsampling (density-agnostic). One Policy to Dress Them All uses voxel-grid filtering + random cropping for robust sim2real; DP3 uses FPS. These reduce sensitivity to sensor density/partiality.”

Yes, voxel-based filtering helps a lot (if what we do is the same as what they do). Other techniques will be taken into account.

“Grasp-synthesis demos are open-loop-optimal, not closed-loop-robust: Grasps from Contact-GraspNet/ACRONYM are static $SE(3)$ targets. Converting them to trajectories for imitation requires a motion generator (motion planner / TAMP as in DemoGen/CP-Gen/MimicGen). Ensure generated approach trajectories are smooth and collision-free and that the pre-grasp approach direction/rotation is consistent — inaccurate approach direction / rotation is a dominant $SE(3)$ failure mode noted in grasp-detection work”

Very true, but how do we inject retry and robustness?

“Delicate grasping needs force/compliance: Vision + point cloud + position/delta-pose control alone cannot regulate contact force. Published evidence (Just Add Force, FORTE) is clear that fragile objects require force feedback or passive compliance. Add current-based/force-torque sensing to observations and (ideally) an impedance/compliant low-level controller or a fast tactile reflex layer beneath the diffusion policy”

I understand, but we just want to see if we can push the limit further to fully vision-based control.

“Sim contact fidelity: Rigid-body sim poorly models compliant/deformable/fragile contact; grasp "success" labels in sim may not reflect real delicate-grasp damage. Validate delicate-grasp quality on real hardware, not just sim success.”

Very true, for this we plan to use deformable simulation (or use force detected on rigid object).

Regarding the ability to learn different grasp poses

Bunny-VisionPro: Real-Time Bimanual Dexterous Teleoperation for Imitation Learning

- The spatial generalization in Figure 8 consists of small range of variations, and it does not show if the policy generates different grasp poses for different object orientations

MimicGen assumes delta-pose action space: Appendix - “N.1 Equivalence between delta-pose actions and controller target poses”

MimicGen tasks that have object rotations

- Stack D1
- Stack Three D1
- Square D2
- Threading D1 D2
- Three Piece Assembly D2
- ...

My observation is that for all these tasks with rotation variation (of course other things vary too), the success rates are quite low.

In SoftMimicGen, on the Franka - Rigid Cube task, diffusion policy performs worse than BC-RNN-GMM. Some other tasks too.